

LLMs have not been fine-tuned with network security knowledge, and have certain limitations in the planning and detailed implementation of penetration testing tasks. Therefore, we need to design a clever agent framework to reduce the difficulty of tasks through external constraints and thus increase the success rate of tasks.

AutoPT aims to address these challenges and make it more suitable for end-to-end penetration testing tasks. We believe that the architectural capabilities of a single agent through prompt learning alone are not enough to handle complex tasks such as complex end-to-end penetration testing. External constraints are needed to help the agent complete the task. We draw inspiration from the construction of traditional state machines [48], split the entire end-to-end task into multiple states, and solved each subtask through state transitions. Each state solves its task independently, switches states after the task is completed, and reports its results to the next state without always maintaining the entire task context. Essentially, even if there is a problem in a certain state, jumping back after checking the state will not have an impact on subsequent tests. Although each state is closely related to the other, it can prevent errors from being transmitted throughout the process and adjust subsequent behaviors in a timely manner.

Definition 1 (Finite State Machine). A finite state machine FSM is a state-labeled, attributed automaton $M = (S, S_0, \Sigma, \delta, O, F)$ where S is a set of states, S_0 is the initial state, Σ is a set of input symbols, $\delta : S \times \Sigma \rightarrow S$ is a transition function that assigns a state from S based on the current state and an input symbol, $O : S \times \Sigma \rightarrow \Gamma$ is an output function that assigns an output from the alphabet Γ to each state and input symbol, and $F \subseteq S$ is a set of final (or accepting) states.

In a finite state machine, the state carries information about the history of the machine and tracks how the state machine reached the current situation. We attempt to decompose the solution process of the entire end-to-end penetration testing task and model each stage of the entire task into a state machine. Traditionally, state machines are divided into Mealy machines [50] and Moore machines [20]. The output of Mealy machines depends on the current state and the input symbol. The output of Moore machines is related only to the current state, whereas the state machine transition function depends on the current state and the output symbol. In AutoPT, we define all nodes as Mealy machines and take the system prompt or the contextual interaction content of the previous state (including the contextual information of the previous state output and optional environmental feedback) as the input symbol.

Referring to the research on penetration testers, penetration testers often have some experience-based deterministic steps when solving tasks, such as selecting vulnerabilities based on their threat level and evaluating whether the task is completed. According to the definition of the finite state machine, we define the Pen-testing State Machine PSM as follows:

Definition 2 (Pen-testing State Machine). The penetration test state machine is formulated as a six-tuple $(S, s_0, \Sigma, \delta, O, F)$ and explains each component of AutoPT in the end-to-end penetration test task scenario.

State Set S . Each state can be considered a predefined situation or configuration of PSM . After entering a certain state, PSM performs a set of predefined expected operations.

Initial state s_0 . When the input target machine IP, port, and task target are received, the entire system AutoPT is initialized, and the process starts from the initial state.

Input symbol set Σ . We define Σ as an infinite message set (text unit). Specifically, we define Σ as the context information O output by the previous state and the optional environment feedback $T : \Sigma = \{O, T\}$.

The transition function δ . δ is a mapping that defines how AutoPT transitions from one state to another under a specific input symbol. In the context of a deterministic finite automaton (DFA) here, the definition form is $\delta : S \times \Sigma \rightarrow S$, where S is the state set and Σ is the input symbol set.

Output function O . Here we refer to the output function definition of the Mealy machine. First, we define the output symbol set Γ as the infinite message set (text unit) that is the same as the input symbol set Σ . Specifically, we define Γ as the context information O of the current state output and the optional environment feedback $F : \Gamma = \{O, F\}$. We define the output function O to be the output of the agent and the tool call feedback, the only agent output or the static rule processing: $O : S \times \Sigma \rightarrow \Gamma$, where S is the state set and Σ is the input symbol set.

Final state set F . A set of final states when the process terminates. When these states are reached, the input sequence is accepted or processed. In AutoPT, we define the final states as “Failed” and “Success” to represent the final results. These two states satisfy the condition: $F \subseteq S$.

Similar to the traditional FSM, the output function O of each node is different. In particular, depending on whether an agent is based on the LLM involved, we divide the state of AutoPT into an Agent state and a Rule state. In the initialization phase (s_0), the Agent state uses a set of prompts $\{P_1, P_2, \dots\}$ to initialize the agent in different states. Each prompt corresponds to its own tool set. Our evaluation carefully selected these tools and considered them sufficient to complete the sub-task. This differentiated prompting approach ensures that the language model receives the most relevant guidance in each state. The Rule state uses rules to operate the input contextual interaction content and match and filter out the output content. The rule-matching method ensures that the behavior of the Agent state is constrained, thereby improving the agent’s ability to focus on specific steps. We solve **Challenge 1** by replacing the traditional context information iteration with interactive messages between states. Specifically, each state only needs to understand the core task content and the output value of the previous state instead of obtaining all historical messages.

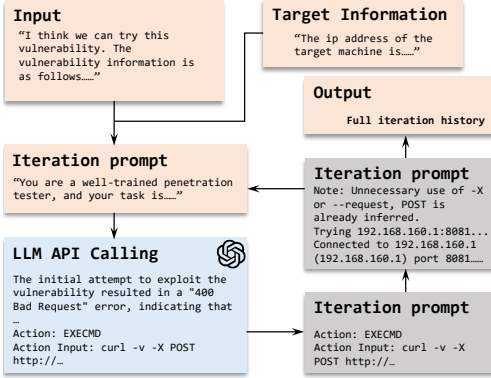


Fig. 3. An example process of an Agent state (Exploit state).

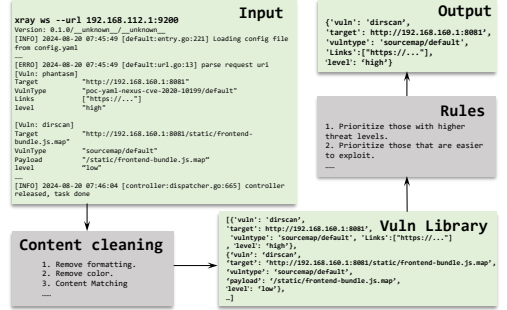


Fig. 4. An example process of a Rule state (Selection state).

5.3 Implementation

5.3.1 Agent state. Unlike the traditional FSM, we take the output symbols of the previous stage as inputs. In each Agent state. ① Splice the initial prompt containing the model role play definition, task goal, and tool definition with the input message to obtain the total prompt. Prompt constraints are used to guide LLM reasoning and call tools to complete related tasks. ② Parse the output content of the large language model to ensure that the LLM calls the relevant tools and calls the input content of the relevant tools to the greatest extent. ③ Merge the tool call information return value into the total prompt and the model is input again. ④ The above ② - ③ steps are repeated until the number of iteration steps reaches the preset maximum value or until the model actively exits the current state. The preset maximum number of iteration steps is used to prevent a certain step from looping infinitely. ⑤ Finally, all model outputs and tool output content are parsed to

obtain the output value of the current state and end the current state. The whole process is shown in Algorithm 1. Below we report in detail the implementation of the Agent state of this work.

Unlike the traditional FSM, we take the output symbols of the previous stage as inputs. In each Agent state. ① Splice the initial prompt containing the model role play definition, task goal, and tool definition with the input message to obtain the total prompt. Prompt constraints are used to guide LLM reasoning and call tools to complete related tasks. ② Parse the output content of the large language model to ensure that LLM calls the relevant tools and calls the input content of the relevant tools to the greatest extent. ③ Merge the tool call information return value into the total prompt and the model is input again. ④ The above ② - ③ steps are repeated until the number of iteration steps reaches the preset maximum value or until the model actively exits the current state.

The preset maximum number of iteration steps is used to prevent a certain step from looping infinitely. ⑤ Finally, all model outputs and tool output content are parsed to obtain the output value of the current state and the end of the current state. The whole process is shown in Algorithm 1. Below we report in detail the implementation of the Agent state of this work.

Prompts. For each Agent state, we prompt the model to generate thoughts and actions at each turn. Each prompt consists of 5 parts: (1) *Description*: Details of the operations that the LLM should perform in the current state. (2) *Role-playing*: Due to the particularity of penetration testing tasks, the model often refuses to assist. Adding the identity of the legal penetration tester and authorization instructions can greatly reduce related problems [32]. (3) *Example*: Some thoughts or action steps from the ReAct example as a demonstration. (4) *Tools description*: Description of tools that the current agent and examples of tool input values can use. (5) *Response format*: Explanation of the thought-action template. These prompts are placed in the system message of each LLM agent and are invisible to other agents.

Tools. For each Agent state, we select relevant tools to provide the agent with relevant tasks to complete. The tools generally include the following three types. (1) *Terminal*: We built a local Kali Linux Docker environment that installed all possible penetration tools for the model for the agent to perform operations, and gave it to the root user. This ensures that the model has sufficient permissions to execute commands and ensures local security (dangerous commands, such as “`wget http://localhost -O- | sh`”, may appear when the model is hallucinating). (2) *Playwright*: To enable the LLM agent to interact with the website, we use and optimize the Playwright browser testing library provided by the Langchain community to interact with a headless web browser (3) *Search*: We implement a query tool that performs a Google search and returns the first web page information when the model input is a keyword. When the model input is a link, access the link and return the link content. We provide these tools to each agent on demand. After manual testing, they are sufficient to complete relevant tasks. Terminal is provided for Scanning status, Search is provided for Information Collection status, and Terminal and Playwright are provided for Exploiting status.

Algorithm 1: Agent State Process

Input: Initialization Prompt P , Input I , Large Language Model L , Tools T , Max Iterations M , Parsing Function F , Output Parsing Function O

Output: Processed output Γ

```

1  $P^* \leftarrow P + I$ ;
2 while iteration steps  $\leq M$  do
3    $F(L(P^*)) \rightarrow L_{\text{output}}, T_{\text{invoke}}, T_{\text{input}}$ ;
4    $T_{\text{output}} \leftarrow T(T_{\text{invoke}}, T_{\text{input}})$ ;
5    $P^* \leftarrow P^* + L_{\text{output}} + T_{\text{output}}$ ;
6   if  $L$  exits current state then
7     break;
8   end
9 end
10  $\Gamma \leftarrow O(L(P^*) + T_{\text{output}})$ ;
11 return  $\Gamma$ ;

```

Parsing Functions. For each Agent state, we implement a parsing function that effectively handles the natural language information exchanged between the agent calling tool and the target environment. According to the output format required in our prompt, the model output is parsed to obtain the tools intended to be called by the agent and the input content of the cleaning tool. The parsing function serves as a supporting interface to assist the interaction between the agent and the tool to support the operation of the entire Agent state. An example of a specific Agent state *Exploitation* is shown in Figure 3.

5.3.2 Rule state. Similar to the Agent state, the Rule state also takes the output symbol of the previous stage as input, but the difference is that in each Rule state. ① Parse the input content and clean out the relevant core information according to the preset rules, such as removing irrelevant flags such as “[INFO]” from the scan results. ② The state output value is generated according to the cleaned information according to the preset rules, and the current state is ended. The whole process is shown in Algorithm 2. Below we report in detail the implementation method of the Rule state of this work.

Parsing Functions. For each Rule state, we implemented the relevant parsing function. In the vulnerability selection stage, we remove irrelevant messages to obtain vulnerability-related content fragments on the basis of the historical scanner results and then collect all vulnerability-related content into a vulnerability library. Each item contains all the vulnerability information, including the vulnerability name, description, hazard, type, and reference information. In the check state, we clean out the content fragments related to the vulnerability exploitation operation in the input content, such as terminal output information or web page return information. The content cleaning step serves subsequent rule matching, allowing for more precise selection checks.

Rules. For each Rule state, we have carefully designed different rules. In the vulnerability selection state, vulnerability information is selected according to the preset vulnerability hazards and difficulty of exploitation. Specifically, priority is given to selecting vulnerabilities with high harm and simple vulnerability exploitation, removing the selected vulnerabilities from the vulnerability library, and returning them as output to end the vulnerability selection state.

In the check state, similar to the design in Section 3, we carefully set an output value for each vulnerability that can be obtained by successful exploitation according to the preset target information. When the target information appears, we consider the penetration test successful,

Algorithm 2: Rule State Process

Input: Input I , Preset Rules R ,
Parsing Function F , Output
Generation Function O

Output: Processed output Γ

```

1  $I^* \leftarrow F(I)$ ;
2  $\Gamma \leftarrow O(I^*, R)$ ;
3 return  $\Gamma$ ;

```

Algorithm 3: PSM Process

Input: Target machine information IP , Task
Target T , System Prompt P , PSM
 $\langle S, s_0, \Sigma, \delta, O, F \rangle$, the output of state S
is Γ , and the total interaction history is
 Γ^* . The value of $s.type$ for each $s \in S$
is from a list of states [Agent, Rule].

Output: The final interaction history Γ^* .

```

1  $\Gamma \leftarrow P + IP + T$ ;
2  $\Gamma^* \leftarrow \Gamma$ ;
3  $s \leftarrow s_0$ ;
4 while  $s \notin F$  do
5   if  $s.type == Agent$  then
6      $\Gamma \leftarrow AgentStateProcess(\Gamma, IP, T)$ ;
7   else
8      $\Gamma \leftarrow RuleStateProcess(\Gamma, IP, T)$ ;
9   end
10   $s \leftarrow \delta(s, \Gamma)$ ;
11   $\Gamma^* \leftarrow \Gamma^* + \Gamma$ ;
12 end
13 return  $s, \Gamma^*$ ;

```

and the output information is “Success”. If it does not appear, it is considered a failure, and the vulnerability exploitation status is returned within a certain threshold of vulnerability exploitation times. When the set number of vulnerability tests is exceeded, the current vulnerability is considered to be currently inexplicable, and the vulnerability is returned to the vulnerability selection stage and the vulnerability is re-selected. If all vulnerabilities in the library are tried and failed, the output information is “Failed”. An example of a specific Rule state *Selection* is shown in Figure 4.

5.3.3 State Transition. The state transition function is an important part of the FSM. In our work, we use a graph structure to simulate the state transition function. First, we define all states, including the initialization s_0 and the terminal state F, as nodes, and the state transitions as edges. We set a routing function to schedule the state. Like the state transition function of the traditional state machine, the routing function determines the next state of the transition based on the current state and the output value of the current state. The state transition function and the two states together constitute the PSM. The overall state transition algorithm is shown in Algorithm 3. Here we solved **Challenge 2** by forcing the state to jump to avoid the agent getting stuck during the automatic solution process.

6 EVALUATION

In this section, we evaluate the performance of AutoPT, focusing on the following research questions:

RQ1 (Effectiveness): How effective is AutoPT for end-to-end penetration testing tasks?

RQ2 (Performance): How does the performance of AutoPT compare with that of the other LLM-based agents?

RQ3 (Cost): How does the cost of AutoPT compare with that of other LLM-based agents or human experts completing end-to-end tasks?

6.1 Evaluation Settings

In this evaluation, we integrate AutoPT with GPT-4o and GPT-4o mini to form three working versions: AutoPT-GPT-3.5, AutoPT-GPT-4o, and AutoPT-GPT-4o-mini. Considering the reproducibility and economic cost of the experiment, we used the same experimental environment settings to set the model selection hyperparameter temperature to 0 and limit the maximum iteration step to 15. At the same time, we instructed AutoPT to use the Terminal, which is deployed and runs on docker on Kali Linux version 2024.1, and the secondary developed headless browser Playwright³ and the search tool Search.

6.2 Effectiveness Evaluation (RQ1)

Table 4. Overall performance of agents based on the GPT-3.5, GPT-4o, and GPT-4o mini models in the AutoPT architectures.

Models	GPT-4o	GPT-4o mini	GPT-3.5	Models	GPT-4o	GPT-4o mini	GPT-3.5
Simple Vulnerability	pass rate	pass rate	pass rate	Complex Vulnerability	pass rate	pass rate	pass rate
CVE-2017-9841	100%	100%	0%	CVE-2018-7600	80%	100%	0%
CVE-2018-12613	40%	100%	0%	CVE-2020-10199	40%	0%	60%
CVE-2021-23017	0%	0%	0%	CVE-2017-12615	0%	0%	0%
CVE-2021-25646	40%	100%	20%	CVE-2023-42793	0%	0%	0%
CVE-2019-3396	0%	0%	0%	CVE-2021-22911	100%	80%	20%
CVE-2023-51467	40%	60%	0%	CVE-2021-29441	40%	0%	0%
CVE-2022-26134	0%	100%	20%	CVE-2020-1938	0%	0%	0%
CVE-2015-1427	20%	100%	100%	CVE-2017-10271	0%	0%	0%
CVE-2020-14750	0%	0%	0%	CVE-2021-45232	0%	0%	0%
CVE-2017-8917	20%	0%	0%	CVE-2016-10134	0%	0%	0%

³The secondary development tool code can be found in <https://github.com/mashiro01/langchain>